SRE Runbook: Web Application Performance Alerting & Incident Management

Problem Statement

Executive Summary

The web application monitoring system is experiencing critical alerting deficiencies that prevent effective incident detection, routing, and remediation across multiple production environments and geographic regions.

Current State Challenges

Challenge 1: Inadequate Alert Granularity

Problem: Alert system generates single consolidated alert for multiple unique application failures across different regions and service components.

Business Impact:

- Delayed incident response due to lack of failure visibility
- Unable to prioritize remediation by region or service criticality
- SLA breaches due to missed individual service failures
- Difficulty in tracking incident metrics per geographic region

Affected Scope:

- **Environments**: Production Web Services (EU-WEST, US-EAST, APAC-SOUTH)
- **Geographic Regions**: UK (London), Germany (Frankfurt), Ireland (Dublin), Singapore (SG), India (Mumbai)
- **Failure Examples** (October 2, 2025):
  - 11:17 AM - UK Region: customer-authentication-service (Response Time: 8500ms)
  - 11:05 AM - Germany Region: payment-gateway-api (HTTP 500 errors)
  - 11:52 AM - Ireland Region: inventory-management-service (Database timeout)
  - 01:45 AM - Singapore Region: checkout-service (API rate limit exceeded)
  - 11:22 AM - UK Region: order-processing-api (Memory leak detected)
  - 01:26 AM - India Region: customer-profile-service (Cache failure)

Sample Alert Data:

timestamp	region	country	service_name	incident_id
10/2/2025, 11:17:11 AM	EU-WEST	UK	customer-authentication-service	auth-8a5f-4e2a-2c04-e3e6
10/2/2025, 11:05:43 AM	EU-WEST	DE	payment-gateway-api	pay-6c1-de5a-ca57-3da7
10/2/2025, 11:52:26 AM	EU-WEST	IE	inventory-management-service	inv-560-de5a-ca57-3da7
10/2/2025, 1:45:50 AM	APAC-SOUTH	SG	checkout-service	chk-268-de5a-ca57-3da7
10/2/2025, 11:22:32 AM	EU-WEST	UK	order-processing-api	ord-96d-4e2a-2c04-e3e6
10/2/2025, 1:26:06 AM	APAC-SOUTH	IN	customer-profile-service	prof-a28-4e2a-2c04-e3e6

Current Behavior: System aggregates all failures → Single remedy ticket created → No region/service context

Expected Behavior: Each unique service failure → Individual remedy ticket → Complete context (timestamp, region, country, service_name, incident_id, error_type)

Challenge 2: Non-Conditional Email Alert Execution

Problem: Email alerting mechanism fires on every health check execution regardless of service health status.

Business Impact:

- Alert fatigue leading to ignored genuine incidents
- Increased operational overhead filtering false positives
- Email system overload with empty/successful health check notifications
- Reduced team confidence in monitoring system
- On-call engineers overwhelmed with non-actionable alerts

Root Cause: Monitoring service task limitation - unable to evaluate conditions before sending; email notification workaround lacks conditional logic to check failure state before execution.

Current Behavior: Health check runs → Email triggers → Email sent (even for successful checks with no issues)

Expected Behavior: Health check runs → Condition evaluated → Email sent ONLY if performance degradation or failure detected

---

SRE Runbook

Section 1: Incident Detection & Classification

1.1 Incident Identification

When to Use This Runbook:

- Service failure alerts not generating individual tickets
- Receiving email alerts with empty/no failure data

- Multiple regions affected but single ticket created
- Unable to identify which specific service/region failed
- Response time degradation not properly alerted
- HTTP error rate spikes not triggering separate alerts

1.2 Severity Classification

Severity	Criteria	Response Time	Example
SEV-1 Critical	3+ regions affected, revenue-impacting services down	15 minutes	Payment gateway down across all regions
SEV-2 High	2 regions affected OR single critical service	1 hour	Authentication service failing in EU
SEV-3 Medium	Single region, non-critical service	4 hours	Recommendation engine slow response
SEV-4 Low	Dev/staging environment only	Next business day	Test environment API timeout

1.3 Service Criticality Matrix

Service Type	Business Impact	Alert Priority	Escalation Required
Payment Gateway	Direct revenue loss	SEV-1	Yes - Immediate
Authentication	User access blocked	SEV-1	Yes - Immediate
Checkout	Order processing failure	SEV-2	Yes - Within 30 min
Inventory	Stock accuracy	SEV-2	Yes - Within 1 hour
Recommendations	User experience	SEV-3	No - Standard process
Analytics	Reporting delay	SEV-4	No - Business hours

Section 2: Immediate Response Actions

2.1 Validate Alert Authenticity

Step 1: Check ServiceNow/Jira Ticket

- Log into incident management portal
- Search for tickets created in last 2 hours with tag "WebApp-Monitoring"
- Review ticket description for service failure details
- Note: If multiple regions affected but only one ticket exists → Confirm aggregation issue

Step 2: Verify Actual Service Health Status

- Access Application Performance Monitoring (APM) Dashboard

    - DataDog: https://app.datadoghq.com/dashboard/web-services
    - New Relic: https://one.newrelic.com/web-applications
    - Prometheus/Grafana: https://monitoring.company.com/grafana
- Navigate to affected region dashboard (e.g., EU-WEST)
- Check service health indicators:
    - Response time metrics
    - Error rate percentages
    - Active user sessions
    - Database connection pools
    - Cache hit rates
- Identify all degraded/failed services in last execution window

Step 3: Cross-Reference Email Alerts

- Check email inbox for monitoring notifications
- Document: Did you receive one email or multiple?
- Check email content: Is failure data present or empty?
- If empty alerts received → Confirm conditional alert issue
- Check email timestamp vs. actual failure time → Validate alert delay

2.2 Verify Customer Impact

Immediate Customer Impact Check:

- Review customer support ticket queue for spike in complaints
- Check social media monitoring tool for service complaints
- Review active user sessions in last 15 minutes vs. baseline
- Check transaction success rate in payment dashboard
- Review web analytics for traffic drop or bounce rate spike

Customer Impact Assessment:

```
CUSTOMER IMPACT TEMPLATE:
--------------------------
Service Affected: [customer-authentication-service]
Region: [UK - London]
Customers Impacted: [~5,000 active users at time of failure]
Business Impact: [Unable to log in, checkout blocked]
Revenue Impact: [Estimated £X per minute of downtime]
Duration: [17 minutes - 11:17 AM to 11:34 AM]
Status: [RESOLVED / IN PROGRESS]
```

2.3 Create Detailed Incident Log

Document in incident ticket:

```
INCIDENT LOG TEMPLATE:
-----------------------
Timestamp: [Current time]
Alert Source: [ServiceNow/Email/APM Tool/Customer Report]
Number of Tickets Created: [1 or multiple]
Number of Actual Service Failures: [List count]

Failed Services Identified:
1. Region: [EU-WEST]
   Country: [UK]
   Service: [customer-authentication-service]
   Incident ID: [auth-8a5f-4e2a-2c04-e3e6]
   Error Type: [Response Time Degradation]
   Metric: [Response time: 8500ms (SLA: 200ms)]
   Timestamp: [10/2/2025, 11:17:11 AM]
   Customer Impact: [High - Login failures]

2. Region: [EU-WEST]
   Country: [DE]
   Service: [payment-gateway-api]
   Incident ID: [pay-6c1-de5a-ca57-3da7]
   Error Type: [HTTP 500 Internal Server Error]
   Metric: [Error rate: 45% (SLA: <1%)]
   Timestamp: [10/2/2025, 11:05:43 AM]
   Customer Impact: [Critical - Payment failures]

3. [Repeat for each failure]

Gap Identified: [Alert aggregation / Empty email alerts]
Workaround Applied: [Manual ticket creation per service]
```

Section 3: Workaround Procedures

3.1 Manual Ticket Creation for Missing Alerts

Purpose: Until alert system fixed, manually create individual tickets for each failed service

Procedure:

1. **Identify All Service Failures** (Check intervals: 9 AM, 2 PM, 6 PM, 11 PM)
   - Access APM dashboard
   - Filter by: Health Status = "Degraded" or "Down", Time Window = Last 8 hours
   - Check key metrics:
     - Response time > SLA threshold
     - Error rate > 5%
     - Availability < 99.9%
   - Export failure list to spreadsheet
2. **Create Individual ServiceNow/Jira Tickets**
   - For each unique failure, create separate incident
   - Use standardized naming: `WebApp Service Degradation - [Region] - [Country] - [Service_Name]`
   - Priority based on Section 1.2 severity matrix
3. **Ticket Template**:

Short Description: WebApp Service Degradation - [UK] - [customer-authentication-service]

Description:
Automated alert failed to generate individual ticket.
Manual ticket creation per SRE runbook.

Environment: Production
Region: [EU-WEST]
Country: [UK - London]
Service Name: [customer-authentication-service]
Incident ID: [auth-8a5f-4e2a-2c04-e3e6]
Failure Timestamp: [10/2/2025, 11:17:11 AM]

Performance Metrics:
- Response Time: 8500ms (SLA: 200ms)
- Error Rate: 12% (SLA: <1%)
- Availability: 94% (SLA: 99.9%)

Customer Impact: High - Users unable to authenticate
Estimated Users Affected: ~5,000 concurrent users

Business Impact: Login failures preventing checkout completion

Immediate Action Required: Investigate authentication service performance degradation

Assignment Group: [Regional Support Team - UK]
Priority: P1 - Critical

4. **Route to Correct Team**:
   - UK failures → UK regional operations team
   - Germany failures → DE regional operations team
   - Ireland failures → IE regional operations team
   - Singapore failures → APAC operations team (consider timezone)
   - India failures → India regional operations team
5. **Set Up Temporary Manual Monitoring Dashboard**:
   - Create shared spreadsheet: "Manual Service Health Tracking"
   - Columns: Timestamp | Region | Country | Service | Status | Ticket ID | Owner
   - Update every monitoring cycle until automation fixed
   - Share with: SRE team, regional ops teams, incident manager

3.2 Email Alert Filtering Process

Purpose: Reduce alert fatigue from empty email notifications

Daily Procedure:

1. **Morning Alert Review** (9:00 AM)
   - Check all monitoring alert emails received overnight
   - Create email folders:
     - "WebApp - Critical Alerts"
     - "WebApp - False Positives"
     - "WebApp - Successful Health Checks"
   - Filter emails based on content:
     - Contains failure data (error rate, response time issues) → Move to "Critical Alerts"
     - Empty/null results or successful health check → Move to "Successful Health Checks"
     - Contains data but metrics within SLA → Archive as "False Positive"
2. **Create Digest Summary**
   - Use spreadsheet to track genuine alerts
   - Share daily summary with team at 9:30 AM standup:

```
WebApp Alert Summary - [October 2, 2025]
==========================================
Total Emails Received: 148
Genuine Service Failures: 6
False Positives (Successful checks): 140
False Positive Rate: 94.6%

Critical Action Items:
1. UK Auth Service - Response time 8500ms - P1 - Owner: John
2. DE Payment Gateway - HTTP 500 errors - P1 - Owner: Sarah
3. IE Inventory Service - DB timeout - P2 - Owner: Michael
4. SG Checkout Service - Rate limit - P2 - Owner: Priya
5. UK Order Processing - Memory leak - P2 - Owner: David
6. IN Customer Profile - Cache failure - P3 - Owner: Raj

Regional Breakdown:
- EU-WEST: 4 incidents
- APAC-SOUTH: 2 incidents

SLA Status: 2 breaches (Auth Service, Payment Gateway)
```

3. **Weekly Escalation**
   - If false positive rate > 80%, escalate to monitoring platform team

- Attach weekly statistics to escalation ticket
- Include business impact: "Team spending X hours/day filtering false alerts"

3.3 Interim Regional Monitoring Schedule

Purpose: Ensure no incidents missed during alert system issues

24/7 Monitoring Rotation:

```
MANUAL MONITORING SCHEDULE:
============================
Shift 1 (00:00 - 08:00 UTC): APAC Team
- Check every 2 hours: 00:00, 02:00, 04:00, 06:00
- Focus regions: APAC-SOUTH, APAC-EAST

Shift 2 (08:00 - 16:00 UTC): EU Team
- Check every 2 hours: 08:00, 10:00, 12:00, 14:00
- Focus regions: EU-WEST, EU-CENTRAL

Shift 3 (16:00 - 00:00 UTC): US Team
- Check every 2 hours: 16:00, 18:00, 20:00, 22:00
- Focus regions: US-EAST, US-WEST

Weekend Coverage: On-call rotation
```

Manual Check Procedure (Complete in 10 minutes):

1. Log into APM dashboard
2. Quick scan all regions - traffic light status
3. Check critical services first (payment, auth, checkout)
4. If any RED status → Create ticket immediately
5. If any YELLOW status → Monitor for 15 minutes, escalate if persists
6. Document findings in shared log

Section 4: Root Cause Investigation

4.1 Alert Aggregation Issue - Investigation Steps

Step 1: Review Alert Configuration

- Contact Monitoring Platform Engineering team
- Schedule 30-minute review meeting within 24 hours
- Bring sample data from Section 2.3 (multiple failures, single ticket)

- Questions to ask:
  - "Are alert labels configured for region and country dimensions?"
  - "What is current alert grouping configuration?"
  - "Is there a de-duplication window configured?"
  - "Can we see the alert routing rules?"
- Request current alert manager configuration documentation
- Request sample alert payload for last incident

Step 2: Check ServiceNow/Jira Integration

- Meet with ITSM team to review webhook configuration
- Questions to explore:
  - "Is webhook configured to create individual tickets or batched tickets?"
  - "Are there any business rules that merge similar incidents?"
  - "What fields from monitoring alerts map to ticket fields?"
  - "Is there a de-duplication rule checking for similar descriptions?"
- Review incident creation workflow in ServiceNow
- Check assignment rules - verify region-based routing exists
- Test webhook manually with sample payload containing multiple distinct failures

Step 3: Analyze Alert Payload Structure

- Request sample alert JSON payload from monitoring system for last aggregated incident
- Review payload structure - verify contains:

```
Expected Fields:
✓ Unique incident identifier per service failure
✓ Region field populated (EU-WEST, APAC-SOUTH, etc.)
✓ Country field populated (UK, DE, IE, SG, IN)
✓ Service name field populated
✓ Error type/category
✓ Metric values (response time, error rate, etc.)
✓ Timestamp per failure
```

- If fields missing → Alert definition needs enhancement
- If fields present but not used → Integration mapping needs fix

Step 4: Test Alert Routing Logic

- Create test scenario in staging environment:

    - Trigger 3 simultaneous failures in different regions
    - Different services in each region
    - Monitor ticket creation behavior
- Expected outcome: 3 separate tickets
- Actual outcome: Document observed behavior
- If single ticket created → Identify where aggregation occurs:
    - In monitoring tool (alert grouping)
    - In webhook (batch processing)
    - In ServiceNow (business rules)

Step 5: Document Findings Create investigation report:

INVESTIGATION REPORT TEMPLATE:

Issue: Alert Aggregation Preventing Individual Ticket Creation

Date Investigated: [October 3, 2025]

Investigated By: [SRE Team Lead]

Investigation Duration: [4 hours]

Current State Analysis:

- Alert Manager Version: [v0.25.0]
- ServiceNow Integration: [REST API webhook]
- Alert Grouping Window: [5 minutes]
- De-duplication Rules: [Active - 10 minute window]

Findings:

☑ Alert labels missing region/country dimensions in Prometheus rules

☑ ServiceNow webhook configured for batch processing (max 10 alerts per request)

☐ Alert grouping interval set too high (>5 minutes) - Current: 5 min (acceptable)

☑ ServiceNow de-duplication rules combining alerts with similar service names

☐ Other issues identified: [None]

Root Cause Identified:

Primary: Alert definitions lack granular labels for region and country

Secondary: ServiceNow business rule "Similar Incident Merge" is over-aggressive

Impact Analysis:

- Incidents per day: ~30
- Tickets created: ~5 (should be 30)
- Missed SLA tracking: 25 incidents daily
- Delayed response: Average 45 minutes per incident

Recommendation:

1. Add region and country labels to all Prometheus alert rules
2. Update ServiceNow webhook to process alerts individually (not batched)
3. Modify ServiceNow business rule to exclude web service alerts from auto-merge
4. Implement alert routing based on region labels

Estimated Effort:

- Monitoring team: 3 days (alert rule updates)
- Integration team: 2 days (webhook modification)
- ITSM team: 1 day (business rule adjustment)

Total: 6 business days

Owner: Monitoring Platform Team Lead

Target Resolution Date: October 17, 2025

Status: In Progress

4.2 Conditional Email Alert Issue - Investigation Steps

Step 1: Understand Current Email Workflow

- Schedule meeting with monitoring platform team
- Document current email alert architecture:
  - What triggers email? (scheduled health check, alert rule, webhook)
  - Where is email configuration? (alert manager, custom script, monitoring tool)
  - What condition logic exists currently? (none, basic, complex)
- Map workflow on whiteboard or diagram tool:

```
Current Flow:
Health Check Runs (Every 5 min) → Email Sends (Always) → Inbox Flooded

Desired Flow:
Health Check Runs → Evaluate Metrics → IF (failure detected) → Email Sends
```

Step 2: Review Monitoring Tool Email Capabilities

- Research monitoring tool documentation:
  - DataDog: Check "Monitor Notifications" conditional logic features
  - New Relic: Review "Alert Conditions" threshold configuration
  - Prometheus: Check Alertmanager routing and inhibition rules
- Question to answer: "Does our monitoring tool support conditional notifications natively?"
- If YES → Configuration issue likely
- If NO → Need workaround or tool migration

Step 3: Identify Why Emails Send Unconditionally Investigate possible causes:

- **Cause A**: Email configured in "always notify" section of health check
  - Fix: Move to "alert conditions" section
- **Cause B**: Email script runs on schedule, not triggered by alert
  - Fix: Change to alert-triggered workflow
- **Cause C**: Alert rule has no threshold, fires on every check
  - Fix: Add proper threshold conditions (e.g., error_rate > 5%)
- **Cause D**: Email routing rule has no filters
  - Fix: Add filters in alert manager routing

Step 4: Evaluate Alternative Solutions Compare options and select best fit:

Option A: Native Tool Conditional Alerts

- Pros: No custom development, officially supported
- Cons: May require tool upgrade, learning curve
- Feasibility: High if tool supports it
- Timeline: 1-2 weeks

Option B: Implement Custom Alert Router

- Pros: Full control, can add complex logic
- Cons: Maintenance overhead, requires development
- Feasibility: Medium - needs engineering resources
- Timeline: 3-4 weeks

Option C: Use Webhook with External Service

- Pros: Flexible, can add enrichment and filtering
- Cons: Additional service dependency
- Feasibility: Medium-High
- Timeline: 2-3 weeks

Option D: Alert Manager Routing Configuration

- Pros: No code changes, configuration only
- Cons: Limited to Alert Manager capabilities
- Feasibility: High
- Timeline: 3-5 days

Step 5: Design Conditional Logic Requirements Document conditions that should trigger email:

```
EMAIL SHOULD SEND WHEN:
========================
Condition 1: Service response time > SLA threshold
  - Authentication: >200ms
  - Payment Gateway: >500ms
  - Checkout: >1000ms
  - Other services: >2000ms

Condition 2: HTTP error rate > 5%
  - Any 5xx errors
  - Any 4xx errors (excluding 401, 404)

Condition 3: Service availability < 99.9%
  - Calculated over 5-minute window

Condition 4: Database connection pool exhausted
  - Available connections < 10%

Condition 5: Memory usage > 85%
  - Sustained for 10 minutes

EMAIL SHOULD NOT SEND WHEN:
=============================
- All health checks pass
- Metrics within SLA thresholds
- Planned maintenance window active
- Test/staging environment (unless configured)
- Duplicate alert within 15 minutes
```

Step 6: Pilot Solution in Staging

- Select lowest-impact service for testing (e.g., analytics-service in staging)
- Implement chosen conditional logic solution
- Test scenarios:
  1. Trigger actual failure → Verify email sent with correct details
  2. Run 20 successful health checks → Verify NO emails sent
  3. Trigger failure during maintenance window → Verify email suppressed
  4. Trigger multiple failures 5 minutes apart → Verify only 1 email sent
- Monitor for 72 hours (3 days)
- Measure metrics:
  - False positive rate: Target < 5%
  - Alert delay: Target < 2 minutes from failure

    - Email content accuracy: Target 100%
- If metrics met → Proceed to production rollout
- If metrics not met → Troubleshoot and iterate

Step 7: Document Root Cause

INVESTIGATION REPORT TEMPLATE:

Issue: Unconditional Email Alerts Causing Alert Fatigue
Date Investigated: [October 3, 2025]
Investigated By: [SRE Monitoring Lead]

Current Configuration:
- Monitoring Tool: [DataDog APM]
- Alert Method: [Email via DataDog Monitors]
- Health Check Frequency: [Every 5 minutes]
- Email Recipients: [15 engineers + 3 distribution lists]

Root Cause Identified:
Email notification configured in "Always Notify" section of health check monitor instead of "Alert Conditions" section. This causes email to send on every check execution regardless of service health status.

Evidence:
- Screenshot of current monitor configuration
- Email logs showing 288 emails/day per service (24h × 12 checks/hour)
- 0% correlation between emails and actual service failures

Impact Analysis:
- Daily false positive emails: 4,320 (15 services × 288 emails/day)
- Engineer time wasted filtering: ~2 hours/day per person
- Critical alerts missed: 3 incidents in last 7 days
- Team confidence in monitoring: Low (survey: 3.2/10)

Solution Selected: Option D - Alert Manager Routing Configuration
- Justification: Fastest implementation, no code required, leverages native tool capabilities

Implementation Plan:
1. Reconfigure monitors to use alert conditions only
2. Set appropriate thresholds per service type
3. Add maintenance window suppression
4. Implement 15-minute de-duplication window
5. Test in staging for 3 days
6. Gradual rollout to production (10% → 50% → 100%)

Owner: Monitoring Platform Team
Target Resolution Date: October 12, 2025
Status: Solution designed, awaiting approval

Section 5: Permanent Fix Implementation

5.1 Alert Granularity Enhancement

Solution Design Requirements:

Each service failure must generate unique alert with labels:

- `region`: Cloud region (EU-WEST, US-EAST, APAC-SOUTH)
- `country`: Geographic country code (UK, DE, IE, SG, IN)
- `service_name`: Exact service name (customer-authentication-service)
- `incident_id`: Unique failure identifier
- `error_type`: Category (response_time, http_error, timeout, etc.)
- `severity`: Impact level (critical, high, medium, low)
- `timestamp`: ISO 8601 format timestamp
- `metric_value`: Actual metric that violated threshold
- `metric_threshold`: SLA threshold that was violated

Alert routing rules must support:

- Region-based assignment (EU-WEST → European ops team)
- Country-specific assignment (UK → London team, DE → Frankfurt team)
- Service-type routing (payment-* → payments team)
- Severity-based prioritization
- SLA tracking per service per region
- Business hours vs. after-hours routing

Implementation Plan:

Phase 1: Alert Definition Enhancement (Week 1-2)

- Monitoring platform team updates all Prometheus/DataDog alert rules
- Add required labels to each alert definition
- Update alert naming convention:

```
Old: WebAppDown
New: WebApp-{region}-{country}-{service}-{error_type}
Example: WebApp-EU-WEST-UK-customer-auth-response_time
```

- Create service-specific thresholds document
- Test updated rules in staging environment

- Validate alert labels appear correctly in Alert Manager

Phase 2: ServiceNow Integration Update (Week 2-3)

- ITSM team updates webhook receiver to handle new labels
- Create field mapping document:

```
Alert Label → ServiceNow Field
---------------------------------
region → custom_field_region
country → custom_field_country
service_name → configuration_item
error_type → custom_field_error_category
severity → impact (1=SEV-1, 2=SEV-2, etc.)
```

- Modify incident creation template to use new fields
- Update assignment group routing logic based on region/country
- Disable old "Similar Incident Merge" business rule for WebApp alerts
- Test webhook with sample payloads containing all new fields

Phase 3: Alert Routing Configuration (Week 3-4)

- Create regional assignment groups if don't exist:
  - WebApp-Support-EU-UK
  - WebApp-Support-EU-DE
  - WebApp-Support-EU-IE
  - WebApp-Support-APAC-SG
  - WebApp-Support-APAC-IN
- Configure assignment rules:

```
IF region = "EU-WEST" AND country = "UK"
  THEN assign to "WebApp-Support-EU-UK"

IF region = "EU-WEST" AND country = "DE"
  THEN assign to "WebApp-Support-EU-DE"

IF service_name LIKE "payment-%"
  THEN assign to "Payments-Team" (regardless of region)
```

- Set up escalation rules for critical services
- Configure after-hours routing to follow-the-sun support model

Phase 4: User Acceptance Testing (Week 4)

- SRE team validates end-to-end flow
- Test scenarios:
  1. Single service failure → Verify 1 ticket created with correct labels
  2. 3 simultaneous failures in different regions → Verify 3 separate tickets
  3. Same service failure in 2 countries → Verify 2 tickets, different assignments
  4. Critical service failure → Verify correct priority and escalation
  5. After-hours failure → Verify routes to on-call team
- Document any issues found
- Sign-off from SRE lead and regional ops leads

Phase 5: Production Rollout (Week 5)

- Gradual rollout strategy:
  - Day 1: Enable for non-critical services (analytics, recommendations)
  - Day 2: Review results, fix any issues
  - Day 3: Enable for medium-critical services (inventory, search)
  - Day 4: Review results
  - Day 5: Enable for critical services (auth, payment, checkout)
- Monitor ticket creation for 48 hours after each phase
- Keep old alerting active as backup during first week
- Complete switchover after validation period

Validation Criteria (Must pass all):

- ✅ Trigger 3 simultaneous failures in UK, DE, IE → 3 separate ServiceNow tickets created
- ✅ Each ticket contains complete context (all required fields populated)
- ✅ Correct team assignment per region/country
- ✅ Ticket priority matches severity classification
- ✅ Alert-to-ticket latency < 2 minutes
- ✅ No duplicate tickets for same failure within 15-minute window
- ✅ Manual ticket creation no longer required

Rollback Plan: If critical issues found during rollout:

1. Disable new alert rules immediately
2. Re-enable old alerting configuration

3. Return to manual ticket creation process
4. Schedule post-mortem within 24 hours
5. Address issues before re-attempting rollout

5.2 Conditional Email Alert Implementation

Solution Design Requirements:

Email alert triggers ONLY when:

- Service health metrics violate SLA thresholds:
  - Response time exceeds service-specific limit
  - Error rate > 5% over 5-minute window
  - Availability drops below 99.9%
  - Resource utilization > 85% (CPU/memory/connections)
- Failure persists for minimum duration (avoid transient blips):
  - Critical services: 2 minutes
  - Non-critical services: 5 minutes
- Not in planned maintenance window
- Not duplicate of recent alert (15-minute suppression window)

Email content must include:

- Clear, actionable subject line with service and region
- Service health summary with key metrics
- Direct link to APM dashboard filtered to affected service
- Runbook link for common failure scenarios
- Recent deployment information (if deployed in last 4 hours)
- Customer impact estimate
- Suggested first response steps
- Escalation contact information
- One-click acknowledge link

Implementation Plan:

Phase 1: Conditional Logic Implementation (Week 1)

- Configure Alert Manager routing with conditions
- Create alert threshold hierarchy:

Service Tier 1 (Critical):
- payment-gateway-api: response_time > 500ms for 2 minutes
- customer-authentication-service: response_time > 200ms for 2 minutes
- checkout-service: error_rate > 1% for 2 minutes

Service Tier 2 (High):
- order-processing-api: response_time >